SIMATS ENGINEERING

Department of Information Technology

DSA01 – Object-Oriented Programming with C++

ASSIGNMENT 1 – CO1

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures.

Student Details	Information
Student Name	K VENKATA VAMSI GOUD
Register Number	192472021
Department / Section	CSE-AI
Academic Year / Semester	2024–2028
Faculty Name	Amirtha S
Nature of Assignment	Individual
Duration	1 Week
Total Marks	50
Date of Submission	10-11-2026
GitHub Repository	DSA01-192472021-Assignment1
Compiler / Standard	g++ version to be verified / C++17

GitHub Repository Link: https://github.com/Vamsigoud07/DSA0115_192472021.git

Note: Complete the blank institutional details and repository URL. Compile and run the programs yourself and attach genuine terminal screenshots.

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max Marks	Marks Awarded	Remarks
1	Object-Oriented Analysis	4		
2	Tokens, Keywords, Identifiers and Constants	3		
3	Data Types and Compatibility	4		
4	Implicit Conversions	4		
5	Operators and Precedence	4		
6	Control Structures	4		
7	Procedural vs Object-Oriented	4		
8	Input and Output	4		
9	Implementation and Testing	4		
10	Expectation vs Execution	4		
11	Final Recommendation	4		
12A	Impact Analysis	2		
12B	Evaluation of Original Design	2		
12C	Adaptation or Redesign	2		
12D	Evidence-Based Defence	2		
	TOTAL	50		

OVERALL REMARKS

Student Signature: _____ Faculty Signature: _____

Program Director: _____

OBJECT-ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 1 – CO1

Assignment: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

Nature: Individual | Duration: 1 Week | Total: 50 Marks | Bloom's level: K5 – Evaluate

SUBMISSION AND IMPLEMENTATION NOTES

- Create repository DSA01-192472021-Assignment1 and folders Task01 to Task12.
- Store source files, input files and genuine output screenshots in each task folder.
- Add README.md with student details, compiler/version, exact compile command, run steps and each task result.
- Commit progressively, preferably at least one meaningful commit per task.
- Compile using: g++ -std=c++17 -Wall -Wextra -pedantic filename.cpp -o program.
- The assignment does not specify the concession percentage or rounding convention. For a complete worked design, this report assumes a 50% student/senior concession applied before peak surcharge, with percentage amounts rounded half-up to the nearest paise.

COMMON SCENARIO INTERPRETATION

Fares depend on the farthest zone: Inner Rs 10, Middle Rs 20 and Outer Rs 30. Peak periods are [08:00, 10:00) and [17:00, 20:00). Therefore 08:00 and 17:00 are peak, while 10:00 and 20:00 are off-peak. A completed trip must leave at least Rs 50 on the card. All monetary calculations use integer paise.

TASK 1 – UNDERSTAND THE PROBLEM AND IDENTIFY THE OBJECTS (4 MARKS)

1.1 Objects, data and behaviour

SmartCard: stores a private balance in paise and a card identifier. Its methods are balance(), canTravel(), deduct() and recharge(). It owns the stored value and enforces the reserve rule.

Passenger: stores identity and category (regular, student or senior). isEligibleForConcession() determines whether the passenger qualifies. This separates passenger eligibility from route pricing.

Journey: stores the farthest zone, hour and minute. isValid() and isPeak() validate the trip and determine its time band.

Zone: a fixed set of named values represented by enum class. This prevents arbitrary zone integers from entering fare logic.

FareTable: stores base fares and pricing percentages. calculateFare() centralises policy and avoids repeated literals.

1.2 Procedural and object-oriented approaches

A procedural program stores balances and fare values in variables and uses free functions such as calculateFare(), canTravel() and deductFare(). It can work, but every caller must remember to check the balance before deduction. In an object-oriented design, SmartCard owns a private balance, Journey stores journey information and FareTable owns fare policy. The object that owns the balance also protects it.

1.3 OOP principles

Encapsulation: SmartCard hides balance_; external code cannot directly assign an invalid balance.

Abstraction: the gate calls calculateFare() and card.deduct() without knowing the internal arithmetic.

Inheritance: StudentPassenger and SeniorPassenger may derive from Passenger when they need specialised eligibility rules. A common Pass base class can support different pass products.

Polymorphism: a FareRule interface with virtual apply() can be implemented by PeakSurcharge, ConcessionDiscount and WeekendDiscount classes.

1.4 Rules / invariants

- After a successful journey, balance must be at least 5000 paise.
- A denied journey must not change the balance.
- Invalid zones, hours and minutes must be rejected.
- Peak surcharge applies only in the two half-open time intervals.
- All money is stored and calculated in integer paise.
- Recharge must be positive; fare and balance must not be negative.

1.5 Procedural alternative and complexity

The procedural alternative uses a global balance, a fare array and free functions. A journey requires a fixed number of lookups, arithmetic operations and comparisons: O(1) time. One card balance is a fixed-size integer: O(1) space per card. OOP does not change these asymptotic costs. It reduces maintenance risk: a changed balance rule is local to SmartCard, while procedural code requires reviewing every function that can modify the shared balance.

TASK 2 – USE TOKENS, KEYWORDS, IDENTIFIERS AND CONSTANTS CORRECTLY (3 MARKS)

2.1 Five kinds of tokens

Keywords: class, enum, constexpr, if, else, return and switch. These are reserved words.

Identifiers: SmartCard, BASE_FARE_PAISE, calculateFare and balance_. These are programmer-defined names.

Constants/literals: 5000, 20, true, 'A' and "Denied". These are fixed numeric, character, Boolean and string values.

Operators: =, +, -, *, /, %, &&, ||, += and <<. These perform operations.

Punctuators: { }, ( ), [ ], ; , and ::. These group and separate program elements.

2.2 #define, const and constexpr

#define performs preprocessor text substitution and has no inherent type or normal C++ scope. const is typed and cannot be modified after initialisation, but its value is not necessarily known at compile time. constexpr is typed and must be a compile-time constant expression when valid. constexpr is preferred for fixed fare values because it is type-checked and can be used in constant expressions. A macro such as #define FARE 10 + 20 used as FARE * 2 expands to 10 + 20 * 2 and gives 50 unless parentheses are added. constexpr normally requires no separate run-time storage either. Runtime cost is effectively the same; constexpr is safer.

2.3 enum versus enum class

```
enum OldZone { Inner, Middle, Outer };
enum class Zone { Inner, Middle, Outer };
```

A plain enum may implicitly convert to int and its enumerator names can enter the surrounding scope. enum class requires qualified names such as Zone::Outer and prevents implicit integer conversion. It is therefore safer for zone identifiers.

2.4 Naming and centralised constants

```
enum class Zone { Inner, Middle, Outer };
constexpr long long BASE_FARE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int CONCESSION_PERCENT = 50; // stated assumption
constexpr long long MIN_BALANCE_PAISE = 5000;
```

Identifiers may contain letters, digits and underscores, cannot start with a digit, are case-sensitive and cannot be keywords. This report uses PascalCase for types, camelCase for functions/variables and UPPER_SNAKE_CASE for constants. Unit suffixes such as _PAISE prevent confusion.

2.5 Macro alternative and complexity

Macros have no type checking or normal scope. Both macros and constexpr values generally have no meaningful per-use runtime cost. The choice affects safety and maintainability, not Big-O running time or memory.

TASK 3 – CHOOSE DATA TYPES AND CHECK TYPE COMPATIBILITY (4 MARKS)

3.1 Type selection

- Fare, balance and recharge: long long in paise, for exact integer money.
- Peak and concession flags: bool.
- Hour, minute and small counters: int.
- Zone and passenger category: enum class.
- Journey totals: long long or uint64_t.
- Single character: char.
- Display-only fractional values: double, but not for fare arithmetic.
- auto: appropriate when the initialiser makes the type obvious.

On a typical 64-bit g++ platform, sizeof(bool)=1, char=1, int=4, unsigned int=4, long long=8, float=4 and double=8 bytes. These are common values, not universal guarantees; print sizeof on the actual compiler.

3.2 Why integer paise

Binary floating-point cannot represent many decimal fractions exactly. For example, 0.1 + 0.2 may be represented as 0.30000000000000004. Repeated additions and equality checks can therefore create rounding defects. Rs 12.35 is exactly 1235 paise as an integer. The Rs 50 minimum can also be checked exactly.

3.3 Type compatibility

- bool and char are normally promoted to int in arithmetic expressions.

- When int and double are combined, int is converted to double.
- 7 / 2 is integer division and gives 3; 7.0 / 2 gives 3.5.
- Mixing signed and unsigned integers may convert the signed operand to unsigned, producing surprising comparisons.

3.4 Limits and network total

```
#include <climits>
#include <iostream>
int main() {
   std::cout << INT_MAX << ' ' << LLONG_MAX << '\n';
}
```

Assuming an average fare of Rs 22, three million journeys collect Rs 66,000,000 = 6,600,000,000 paise. This exceeds the common 32-bit INT_MAX of 2,147,483,647. Use long long for the accumulated paise.

3.5 double alternative

A double addition and a long long addition are both O(1), and both commonly occupy 8 bytes. However, double does not guarantee exact paise representation. Error depends on values, addition order and number of journeys. A repeatable experiment should compare the double sum with the exact integer-paise sum at high precision. Integer paise remains the correct financial representation.

TASK 4 – CONTROL IMPLICIT CONVERSIONS (4 MARKS)

4.1 Conversion points and safe practices

- double value = 7 / 2; performs integer division first and stores 3.0.
- Use static_cast<double>(7) / 2 when fractional division is intended for a non-money calculation.
- Comparing signed int with unsigned int may convert the signed value to unsigned; -Wall -Wextra can warn about signedness.
- Brace initialisation rejects narrowing: int count{3.7}; is a compile-time error.
- Use explicit, range-checked conversions for input values and enum indices.

4.2 Predict-the-output program

Predicted output (write this before running the program):

3 3.5

C 68

1 true

s >= u

Explanation: a / b is integer division, so c receives 3. The cast makes d use floating-point division and gives 3.5. 'A' + 2 produces C; printing ch + 1 promotes char to int and prints 68. Assigning 5 to bool produces true; default output is 1, and boolalpha prints true. In s < u, -1 is converted to unsigned for the comparison, becoming a large positive value, so the false branch prints s >= u. Actual output should match the prediction. A compiler commonly emits -Wsign-compare for the signed/unsigned comparison.

4.3 Explicit-conversion alternative

```
int whole{7};
double fractional = static_cast<double>(whole) / 2;
int safeValue{3};
// int invalid{3.7}; // rejected as narrowing
int signedValue = -1;
unsigned int unsignedValue = 1;
bool result = static_cast<long long>(signedValue) <
        static_cast<long long>(unsignedValue);
```

Explicit casts and brace initialisation make conversions visible and catch narrowing at compile time. Built-in casts normally do not change asymptotic time or memory; they improve correctness and clarity.

TASK 5 – WRITE CORRECT EXPRESSIONS WITH THE RIGHT PRECEDENCE (4 MARKS)

5.1 Expressions for peak, surcharge and concession

```
int minutes = hour * 60 + minute;
bool peak = (minutes >= 8 * 60 && minutes < 10 * 60) ||
        (minutes >= 17 * 60 && minutes < 20 * 60);
if (peak) fare += (fare * PEAK_SURCHARGE_PERCENT + 50) / 100;
if (concession) fare -= (fare * CONCESSION_PERCENT + 50) / 100;
```

Multiplication is evaluated before addition. Relational comparisons are evaluated before &&, and && before ||. Parentheses show the two separate peak windows. The surcharge multiplies before dividing and adds 50 for half-up rounding on non-negative paise. Compound assignment updates fare after its right-hand expression is evaluated.

5.2 Increment and negative operands

x++ yields the old value and increments afterwards; ++x increments first and yields the new value. Avoid i = i++; it is confusing and can be undefined in older language standards. Write i++ as a separate statement. Since C++11, integer division truncates toward zero: -10 / 3 is -3 and -10 % 3 is -1.

5.3 Predict-the-output program

Predicted output:

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Explanation: 5 + 3*2 gives 11; (5+3)*2 gives 16. In x++ + ++y, old x=5 and incremented y=4 are added, giving z=9, x=6 and y=4. The conditional returns x=6 because 6>4. (6%4)*2 gives 4. Division and remainder yield 3, 1, -3 and -1. The loop adds 0+3+6+9=18. Actual output should match.

5.4 Alternative: lookup table

```
bool peakTable[24]{};
for (int h = 0; h < 24; ++h)
    peakTable[h] = (h >= 8 && h < 10) || (h >= 17 && h < 20);
bool peakForHour = peakTable[hour];
```

Both the expression and lookup take O(1) time. The table uses 24 Boolean entries (commonly 24 bytes; verify sizeof(bool)). It works for whole-hour rules but cannot alone distinguish 17:00 from 17:30 if a minute boundary is introduced. The expression uses no table storage and handles minute-level conditions.

TASK 6 – DESIGN THE CONTROL STRUCTURES (4 MARKS)

6.1 Choosing control structures

Use switch for the finite zone set and its base fares. Use if–else for time ranges, concession eligibility and payment decisions. Use ?: only for a short two-value expression; avoid nested conditional expressions for fare logic. Use range-based for when processing a collection, indexed for when an index is needed, and while when the number of input records is unknown. Use break to stop switch fall-through; use continue to skip an invalid journey and process the next one.

If no switch case matches, no case body executes. Initialise the fare to an invalid sentinel and reject it; otherwise an unhandled zone could accidentally receive a zero fare.

6.2 Algorithmic pseudocode

```
ALGORITHM CalculateFare
INPUT: zone, hour, minute, concession
OUTPUT: fare in paise or INVALID
BEGIN
  IF hour < 0 OR hour > 23 OR minute < 0 OR minute > 59 THEN
    RETURN INVALID
  END IF
  SWITCH zone
    CASE Inner:  fare <- 1000
    CASE Middle: fare <- 2000
    CASE Outer:  fare <- 3000
```

```
    OTHERWISE: RETURN INVALID
  END SWITCH
  IF concession THEN
    fare <- fare - ROUND_HALF_UP(fare * 50 / 100)
  END IF
  IF (08:00 <= time < 10:00) OR (17:00 <= time < 20:00) THEN
    fare <- fare + ROUND_HALF_UP(fare * 20 / 100)
  END IF
  RETURN fare
END
```

6.3 Completed TODO program

```
#include <iomanip>
#include <iostream>
enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak) {
  long long base = 0;
  switch (z) {
    case Zone::Inner:  base = BASE_PAISE[0]; break;
    case Zone::Middle: base = BASE_PAISE[1]; break;
    case Zone::Outer:  base = BASE_PAISE[2]; break;
    default: return -1;
  }
  if (peak) base += (base * PEAK_SURCHARGE_PERCENT + 50) / 100;
  return base;
}
bool canTravel(long long balance, long long f) {
  return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}
int main() {
  long long balance = 10000;
  struct Trip { Zone zone; bool peak; };
  Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true},
```

```
            {Zone::Middle, false}};
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
                      << std::setfill('0') << f % 100 << std::setfill(' ');
        } else std::cout << "Denied (low balance)";
        std::cout << " balance Rs " << balance / 100 << '\n';
    }
}
```

Expected output:

Fare Rs 10.00 balance Rs 90

Fare Rs 36.00 balance Rs 54

Denied (low balance) balance Rs 54

Trace: Rs 100 − Rs 10 = Rs 90; then Rs 36 leaves Rs 54. The next Rs 20 would leave Rs 34, below the required Rs 50, so it is denied. The supplied fare(Zone,bool) receives peak status from its caller; use a separate isPeak(hour,minute) function to test exact boundaries. 10:00 and 20:00 are off-peak.

6.4 Array alternative

A checked array indexed by a validated zone index also gives O(1) lookup and stores one fare per zone. A switch is explicit and can warn when an enum case is omitted; an unchecked array index can go out of bounds. A fourth zone requires updating the enum, table and validation together. std::array plus bounds checking is safer than a raw array.

TASK 7 – FROM PROCEDURAL TO OBJECT-ORIENTED (4 MARKS)

7.1 Procedural implementation

```
long long balance = 10000;
constexpr long long MIN_BALANCE = 5000;
bool canTravel(long long current, long long f) {
    return f >= 0 && current - f >= MIN_BALANCE;
}
void deduct(long long& current, long long f) {
    current -= f; // unsafe if caller skips canTravel()
}
```

The procedural style is short, but any caller can invoke deduct() without checking. A careless deduction can break the minimum-balance rule.

7.2 Object-oriented implementation

```
class SmartCard {
private:
   long long balance_;
   static constexpr long long MIN_BALANCE = 5000;
public:
   explicit SmartCard(long long initial) : balance_(initial >= 0 ? initial : 0) {}
   long long balance() const { return balance_; }
   bool canTravel(long long f) const {
      return f >= 0 && balance_ - f >= MIN_BALANCE;
   }
   bool deduct(long long f) {
      if (!canTravel(f)) return false;
      balance_ -= f;
      return true;
   }
   bool recharge(long long amount) {
      if (amount <= 0) return false;
      balance_ += amount;
      return true;
   }
};
```

balance_ is private, and deduct() checks the rule immediately before subtracting. recharge() rejects non-positive amounts. A production version should also check overflow before adding a recharge.

7.3 Public-balance alternative

A public balance permits any function to set it to -500 or subtract without checking. Runtime and memory remain O(1), but encapsulation is lost and the minimum-balance invariant can be broken. Both procedural and OOP operations are O(1), and a balance uses one long long value.

TASK 8 – READ AND PRINT DATA SAFELY (4 MARKS)

8.1 Stream error handling

If the operator enters letters where a number is expected, cin >> value fails and sets the stream fail state. Check the extraction result, call cin.clear() to reset the state, then cin.ignore(numeric_limits<streamsize>::max(), '\n') to discard the rest of the invalid line. Without recovery, later extractions also fail.

8.2 Safe input and money formatting

```
#include <iostream>
#include <iomanip>
```

```
#include <limits>
bool readInt(const char* prompt, int& value) {
   std::cout << prompt;
   if (std::cin >> value) return true;
   std::cin.clear();
   std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
   return false;
}
void printMoney(long long paise) {
   std::cout << "Rs " << paise / 100 << '.'
             << std::setw(2) << std::setfill('0') << paise % 100
             << std::setfill(' ');
}
```

setw() sets a minimum field width; setfill() chooses padding. fixed and setprecision(2) are useful for display-only floating-point values. For fares, integer quotient and remainder can print rupees and paise without converting the amount.

8.3 getline() and stoi() alternative

```
std::string line;
if (std::getline(std::cin, line)) {
   try {
      std::size_t used = 0;
      int value = std::stoi(line, &used);
      if (used != line.size()) throw std::invalid_argument("extra characters");
      // apply the required range check
   } catch (const std::exception&) {
      std::cout << "Invalid input line\n";
   }
}
```

Both methods take O(L) time for a line of L characters. getline() stores the line and needs O(L) temporary space. Stream extraction can process tokens without storing the whole line but needs clear() and ignore() for recovery. getline() with stoi() isolates errors by line; checking the consumed character count is essential because stoi("7x") otherwise accepts 7.

TASK 9 – IMPLEMENT AND TEST THE FARE LOGIC (4 MARKS)

9.1 Test strategy

The test set must include all zones in peak and off-peak periods, each time boundary, minimum-balance acceptance, invalid zone/time, negative recharge and non-numeric input. Expected fare is calculated by hand before comparing it with the program.

Test cases:

T1: Inner at 07:59 -> Rs 10.00 (off-peak).

T2: Inner at 08:00 -> Rs 12.00 (peak starts inclusive).

T3: Middle at 10:00 -> Rs 20.00 (peak ends exclusive).

T4: Middle at 17:00 -> Rs 24.00 (evening peak starts inclusive).

T5: Outer at 20:00 -> Rs 30.00 (evening peak ends exclusive).

T6: Outer at 19:59 -> Rs 36.00 (Outer peak).

T7: Balance Rs 62 and fare Rs 12 -> accepted; remaining balance Rs 50.

T8: Unknown zone -> rejected as invalid.

T9: Recharge -Rs 5 -> rejected; balance unchanged.

T10: Hour entered as abc -> input rejected and record skipped or prompt repeated.

9.2 Five-journey manual calculation

Journey 1: Inner, 07:30 -> Rs 10.00; running total Rs 10.00.

Journey 2: Middle, 08:30 -> Rs 24.00; running total Rs 34.00.

Journey 3: Outer, 10:00 -> Rs 30.00; running total Rs 64.00.

Journey 4: Inner, 17:15 -> Rs 12.00; running total Rs 76.00.

Journey 5: Middle, 20:00 -> Rs 20.00; running total Rs 96.00.

Total = Rs 96.00 = 9600 paise. This is for a regular passenger; test concession separately because the assignment does not state its rate.

9.3 Table-driven testing alternative

```
struct TestCase { Zone zone; int hour; int minute; long long expected; };
const TestCase tests[] = {
    {Zone::Inner, 7, 59, 1000},
    {Zone::Inner, 8, 0, 1200},
    {Zone::Middle, 10, 0, 2000},
    {Zone::Middle, 17, 0, 2400},
    {Zone::Outer, 20, 0, 3000}
};
for (const auto& test : tests) {
    // calculate actual fare and compare with test.expected
}
```

For t tests, the loop takes O(t) time and the test table occupies O(t) space. A new boundary test is added as a row, making the suite easier to extend.

TASK 10 – COMPARE EXPECTATIONS WITH EXECUTION (4 MARKS)

10.1 Daily collection estimate with explicit assumptions

The assignment gives journey counts but not the actual distribution by zone, time or concession. For a transparent estimate, assume regular passengers, one-third of journeys in each zone and half of all journeys in peak periods. Average off-peak fare is (10+20+30)/3 = Rs 20. With half the trips receiving a 20% surcharge, the overall average is Rs 22.

20,000 journeys: Rs 440,000 = 44,000,000 paise.

500,000 journeys: Rs 11,000,000 = 1,100,000,000 paise.

3,000,000 journeys: Rs 66,000,000 = 6,600,000,000 paise.

These are model estimates, not measured metro revenue. Actual collection depends on passenger mix. A production system must sum each actual integer-paise fare.

10.2 int, long long and double

A common 32-bit int cannot hold 6,600,000,000 paise; this exceeds INT_MAX. Signed integer overflow is undefined behaviour in C++. long long safely holds the estimated total and is recommended for accumulation. double can hold the magnitude but does not guarantee exact paise representation. Each addition is O(1); common sizes are 4 bytes for int and 8 bytes for long long and double.

10.3 Compiler warnings and evidence

Compile with -Wall -Wextra -pedantic and record the exact compiler version, warnings, output and elapsed time. A signed/unsigned comparison warning indicates a potentially surprising conversion. An unused-variable warning indicates declared data that is not used. Fix warnings rather than suppressing them. Do not claim benchmark timings until measured on your own computer.

TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN (4 MARKS)

11.1 Final design recommendation

Use enum class for zones and passenger categories. Store all money in long long integer paise. Keep fare rates and minimum-balance policy in one constants/configuration module. Use Journey for journey data, Passenger for category, FareTable/FarePolicy for pricing and SmartCard for balance management. Validate input before calculating or deducting a fare. Use table-driven tests for normal, boundary and invalid cases.

11.2 Revisions and limitations

A fare revision should be made in the central fare policy, not by editing repeated literals. A switch is clear for three zones but needs another enumerator and case for a fourth zone. A checked data table is easier to extend. The assumed concession percentage and rounding rule must be confirmed by the metro authority before deployment.

11.3 Whole-network suitability

Fare calculation is O(1) per journey, so N journeys require O(N) total processing time. Per-card state is O(1); storing N journey records is O(N). long long supports the estimated daily total. A production network must additionally handle concurrent gate transactions, atomic balance updates, card authentication, persistent records, audit logs and recharge overflow checks.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN (8 MARKS)

12A. Impact analysis (2 marks)

Add Zone::Airport and a fourth base fare of 6000 paise (Rs 60). Add a 10% weekend discount and state explicitly that it replaces, rather than combines with, the peak surcharge. Add MonthlyPass with validity dates and two selected zones; covered journeys are unlimited while the pass is valid. Update fare-table size, zone validation, fare calculation, reporting and tests. The gate must check pass eligibility before deducting stored-value fare.

12B. Evaluate the original design (2 marks)

The switch can support Airport by adding another case, but repeated branches become less convenient as rules grow. A central fare table is more extensible. SmartCard remains suitable for stored-value transactions, but monthly-pass validity and zone coverage should belong to a separate MonthlyPass class.

Worked examples: Airport weekday off-peak = Rs 60.00; Airport weekday peak = Rs 72.00; Airport weekend = Rs 54.00. If a weekend trip occurs during a peak clock window, it remains Rs 54.00 because the discount replaces the surcharge. A valid pass journey in one of its selected zones costs Rs 0 at the gate.

12C. Revised design and implementation (2 marks)

```
#include <array>
#include <optional>
enum class Zone { Inner, Middle, Outer, Airport };
constexpr std::array<long long, 4> BASE = {1000, 2000, 3000, 6000};
constexpr int PEAK_RATE = 20;
constexpr int WEEKEND_DISCOUNT = 10;
long long roundedPercent(long long amount, int rate) {
   return (amount * rate + 50) / 100;
}
int zoneIndex(Zone z) { return static_cast<int>(z); }
std::optional<long long> calculateFare(Zone z, bool peak, bool weekend) {
   int i = zoneIndex(z);
   if (i < 0 || i >= static_cast<int>(BASE.size())) return std::nullopt;
   long long amount = BASE[static_cast<std::size_t>(i)];
   if (weekend) amount -= roundedPercent(amount, WEEKEND_DISCOUNT);
   else if (peak) amount += roundedPercent(amount, PEAK_RATE);
   return amount;
}
class MonthlyPass {
   int zoneA_, zoneB_, validFrom_, validTo_;
public:
   MonthlyPass(int a, int b, int from, int to)
```

```
    : zoneA_(a), zoneB_(b), validFrom_(from), validTo_(to) {}
  bool covers(Zone z, int day) const {
    int i = zoneIndex(z);
    return day >= validFrom_ && day <= validTo_ &&
        (i == zoneA_ || i == zoneB_);
  }
};
```

This is a compact design sketch. A production version should use real calendar dates, validate that the two pass zones are distinct and valid, and implement purchase, renewal and expiry. std::optional indicates an invalid zone instead of silently returning a zero fare.

12D. Evidence-based defence (2 marks)

D1: Airport weekday off-peak -> Rs 60.00 (new zone base fare).

D2: Airport weekday peak -> Rs 72.00 (20% surcharge).

D3: Airport weekend off-peak -> Rs 54.00 (10% discount).

D4: Airport weekend during peak clock -> Rs 54.00 (no stacking).

D5: Valid pass, covered zone and valid date -> Rs 0 at gate.

D6: Valid pass but uncovered zone -> normal fare.

D7: Expired pass -> normal fare.

Decision: MODIFY the original design. Retain SmartCard encapsulation and integer-paise accounting, replace the growing switch with a checked fare table, make discount selection a separate rule, and add MonthlyPass as an independent object. This preserves original behaviour while separating new policies. The test table lists expected results; attach actual run output and measured timings from your own execution.

FINAL CONCLUSION

The recommended fare system uses typed constants, enum class, integer paise, validated input, explicit peak boundaries and an encapsulated SmartCard. The original design is adequate for three zones; the fourth zone, weekend discount and monthly pass justify a controlled modification. Correctness is supported by boundary calculations and a repeatable test plan. The final submission must include the student's own execution screenshots and working GitHub repository link.

APPENDIX – SUGGESTED GITHUB REPOSITORY STRUCTURE

DSA01-192472021-Assignment1/

|-- README.md

|-- Task01/

|-- Task02/

|-- Task03/

|-- Task04/

|-- Task05/

|-- Task06/

|-- Task07/

|-- Task08/

|-- Task09/

|-- Task10/

|-- Task11/

|-- Task12/

  |-- revised_fare.cpp

  |-- tests.txt

  |-- output-screenshot.png

Suggested compile command: g++ -std=c++17 -Wall -Wextra -pedantic Task06/fare.cpp -o fare

Before submission, compile each program separately, resolve diagnostics, capture genuine outputs, fill missing cover-page information and add the GitHub URL.

APPENDIX – ACTUAL C++ COMPILATION AND RUN OUTPUT

Execution environment: g++ (Debian 14.2.0-19) 14.2.0. Programs compiled with **g++ -std=c++17 -Wall -Wextra -pedantic** and executed. Screenshots show captured standard output and any compiler warning.

Task 4 deliberately includes the signed/unsigned comparison discussed in the answer. The compiler emits the expected -Wsign-compare warning; compilation succeeds.

Task 3 – Integer limits

```
Task 3 — Integer limits
$ g++ -std=c++17 -Wall -Wextra -pedantic task03_limits.cpp -o program
[compile: success]
$ ./program
2147483647 9223372036854775807
```

Task 4 – Type conversions

```
Task 4 — Type conversions
$ g++ -std=c++17 -Wall -Wextra -pedantic task04_conversions.cpp -o program
[compile: success]
Compiler warning:
/mnt/data/compiled_evidence_vamsi/task04_conversions.cpp: In function ‘int main()’:
/mnt/data/compiled_evidence_vamsi/task04_conversions.cpp:8:16: warning: comparison of integer expressions of different signedness: ‘int’ and ‘un
    8 |  if(signedValue<unsignedValue) std::cout<<"s < u\n"; else std::cout<<"s >= u\n";
      |     ~~~~~~~~~~~^~~~~~~~~~~~~~
$ ./program
3 3.5
C 68
1 true
s >= u
```

Task 5 – Operators and precedence

```
Task 5 — Operators and precedence
$ g++ -std=c++17 -Wall -Wextra -pedantic task05_operators.cpp -o program
[compile: success]
$ ./program
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Task 6 – Fare and balance control

```
Task 6 — Fare and balance control
$ g++ -std=c++17 -Wall -Wextra -pedantic task06_fare.cpp -o program
[compile: success]
Compiler warning:
/mnt/data/compiled_evidence_vamsi/task06_fare.cpp: In function ‘long long int fare(Zone, bool)’:
/mnt/data/compiled_evidence_vamsi/task06_fare.cpp:10:2: warning: this ‘if’ clause does not guard... [-Wmisleading-indentation]
   10 |  if(peak) base+=(base*PEAK_SURCHARGE_PERCENT+50)/100; return base; }
      |  ^~
/mnt/data/compiled_evidence_vamsi/task06_fare.cpp:10:55: note: ...this statement, but the latter is misleadingly indented as if it were guarded
   10 |  if(peak) base+=(base*PEAK_SURCHARGE_PERCENT+50)/100; return base; }
      |                                                       ^~~~~~
$ ./program
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

Task 12 – Revised fare design

```
Task 12 – Revised fare design
$ g++ -std=c++17 -Wall -Wextra -pedantic task12_redesign.cpp -o program
[compile: success]
$ ./program
Airport weekday off-peak: Rs 60.00
Airport weekday peak: Rs 72.00
Airport weekend off-peak: Rs 54.00
Airport weekend during peak clock: Rs 54.00
```